PORTFOLIO CYBERSÉCURITÉ — PROJET 02

Infrastructure durcie as-code

Durcissement CIS automatisé · Segmentation réseau · Preuve de conformité Lynis

Ansible · ufw · auditd · Lynis · Hyper-V · Debian 12 · Terraform (AWS)

Styve Landry SIME KOUNOU
styvelandry1@outlook.fr
github.com/landrysime/02-infra-durcie-terraform-ansible
Octobre 2026

1. Contexte & objectif

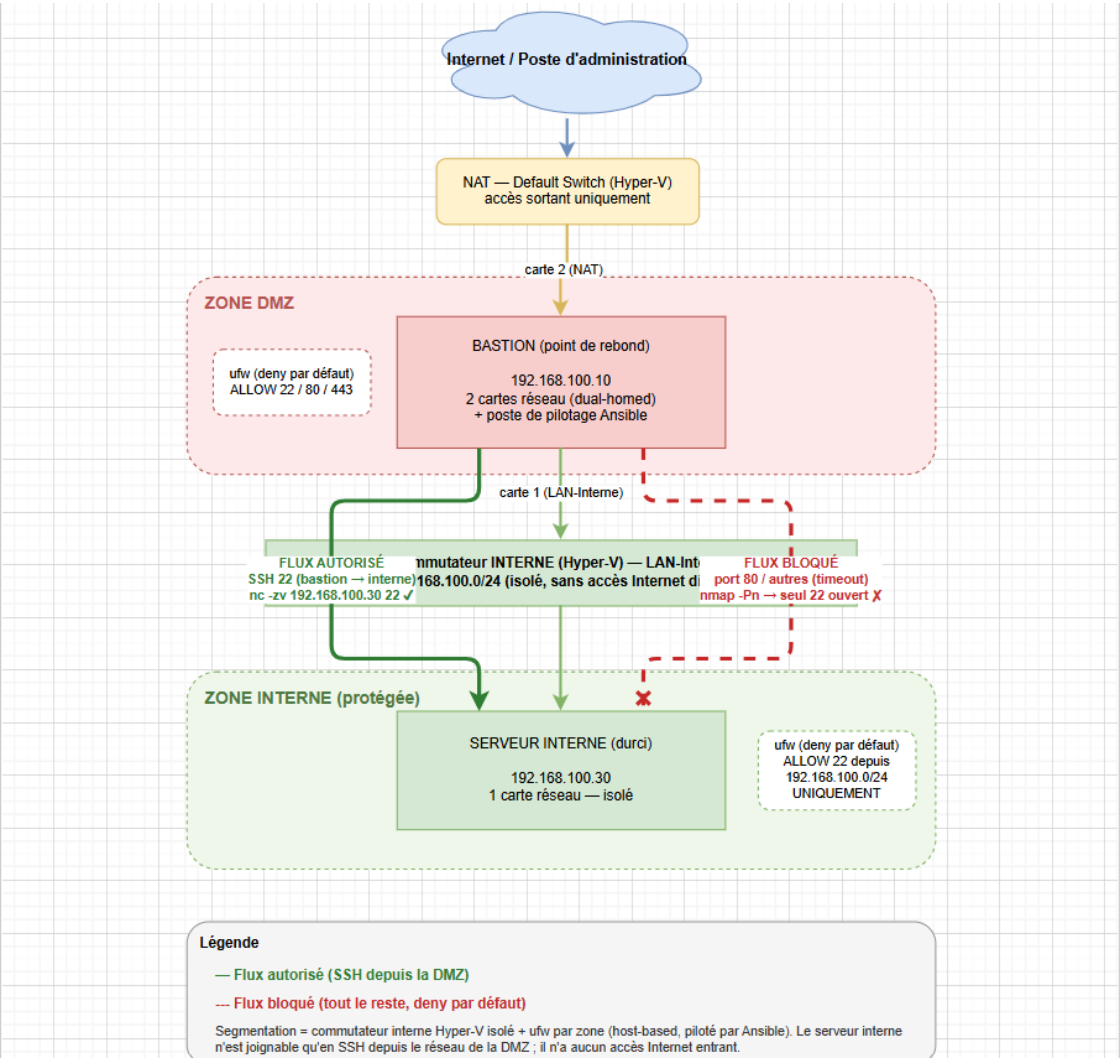
Une entreprise migre son infrastructure, gérée manuellement, vers une approche **Infrastructure as Code**. Le RSSI exige une **segmentation réseau** (DMZ), un durcissement conforme à un référentiel reconnu — le **CIS Benchmark** — et un **rapport de conformité chiffré avant / après**. Le durcissement doit être **reproductible** et **idempotent**.

Ce projet répond à ces exigences avec un durcissement entièrement automatisé par Ansible, une architecture segmentée bastion/DMZ + zone interne, et une mesure de conformité objective (indice de durcissement Lynis) avant et après intervention.

Architecture cible

Machine	Zone	Rôle	Réseau
bastion	DMZ	Point d'entrée exposé + poste de pilotage Ansible	2 cartes (LAN-Interne + NAT)
interne	interne	Serveur protégé, durci	1 carte (LAN-Interne), isolé

Schéma des flux :



Segmentation DMZ / zone interne — flux autorisé (vert) et bloqué (rouge)

2. Pourquoi la démarche a évolué

L'énoncé initial prévoyait un lab **AWS** avec un pare-feu **pfSense**. L'**objectif de sécurité a été tenu à 100 %**, mais deux briques techniques ont été remplacées par des équivalents, pour des contraintes réelles assumées — un réflexe de terrain : garder l'objectif, adapter l'outil.

Virage	Pourquoi	Solution retenue
AWS → Hyper-V	Le compte AWS « Free Plan » interdit EC2 (Service Control Policy) ; l'upgrade exigeait une carte bancaire, volontairement non fournie (maîtrise du risque de facturation).	Lab réalisé sur Hyper-V (local, gratuit). Le code Terraform est conservé et validé (terraform plan) comme preuve de la compétence IaC cloud.
pfSense → ufw par zone	Plus assez de ressources sur le poste pour une 3e VM pfSense.	Segmentation par commutateur interne Hyper-V + filtrage ufw par zone piloté par Ansible (host-based, as-code et idempotent).

Bénéfice : le pare-feu devient du code versionné (cohérent avec l'esprit IaC), et la segmentation reste prouvée par des tests de flux.

3. Déroulé du projet (preuves à l'appui)

Le projet a été mené en 5 phases ; chacune est appuyée par une capture d'écran horodatée.

Phase 1 — Poste de pilotage & IaC cloud (bonus Terraform)

Le poste Windows a été équipé de WSL (Linux sous Windows) puis de Terraform. Le code Terraform décrit la même architecture segmentée côté AWS (VPC, sous-réseaux DMZ/privé, passerelle, security groups) et a été validé par `terraform plan` (`Plan: 8 to add`).



```
deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
styve@Landry-SK:/mnt/c/Windows/System32$ sudo apt update && sudo apt install terraform -y
Get:1 https://apt.releases.hashicorp.com resolute InRelease [12.9 kB]
Get:2 https://apt.releases.hashicorp.com resolute/main amd64 Packages [326 kB]
Hit:3 http://security.ubuntu.com/ubuntu resolute-security InRelease
Ign:4 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:5 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:6 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute InRelease
Fetched 339 kB in 2min 7s (2678 B/s)
All packages are up to date.
Installing:
  terraform

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 35.9 MB
  Space needed: 121 MB / 1024 GB available

Get:1 https://apt.releases.hashicorp.com resolute/main amd64 terraform amd64 1.16.5-1 [35.9 MB]
Fetched 35.9 MB in 1s (33.8 MB/s)
Selecting previously unselected package terraform.
(Reading database ... 35991 files and directories currently installed.)
Preparing to unpack .../terraform_1.16.5-1_amd64.deb ...
Unpacking terraform (1.16.5-1) ...
Setting up terraform (1.16.5-1) ...
styve@Landry-SK:/mnt/c/Windows/System32$ terraform --version
Terraform v1.16.5
on linux_amd64
styve@Landry-SK:/mnt/c/Windows/System32$
```

```
styve@Landry-SK:/mnt/c/Windows/System32$ cd ~
mkdir -p projet02-infra-durcissement/{terraform,ansible,docs}
cd projet02-infra-durcissement
ls
ansible  docs  terraform
styve@Landry-SK:~/projet02-infra-durcissement$ cd terraform
styve@Landry-SK:~/projet02-infra-durcissement/terraform$ cat > main.tf << 'EOF'
terraform {
  required_version = ">= 1.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
EOF
styve@Landry-SK:~/projet02-infra-durcissement/terraform$ cat > variables.tf << 'EOF'
variable "aws_region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-3"
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "projet02-infra"
}

variable "environment" {
  description = "Environnement"
  type        = string
  default     = "dev"
}
EOF
styve@Landry-SK:~/projet02-infra-durcissement/terraform$ terraform init
Initializing the backend...

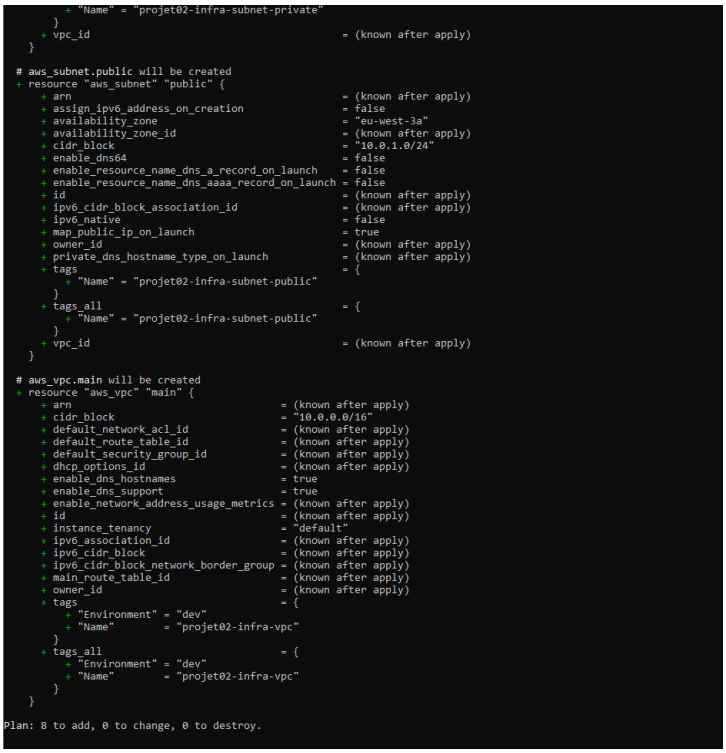
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
styve@Landry-SK:~/projet02-infra-durcissement/terraform$
```

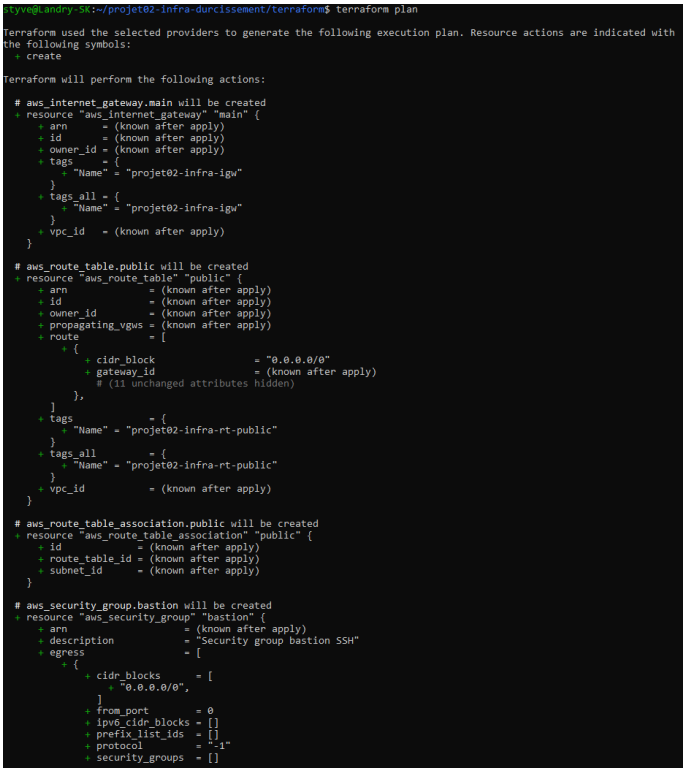


```
      + "Name" = "projet02-infra-subnet-private"
    }
  + vpc_id                                      = (known after apply)
}

# aws_subnet.public will be created
+ resource "aws_subnet" "public" {
  + arn                                         = (known after apply)
  + assign_ipv6_address_on_creation             = false
  + availability_zone                           = "eu-west-3a"
  + availability_zone_id                        = (known after apply)
  + cidr_block                                  = "10.0.1.0/24"
  + enable_dns64                                = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                                          = (known after apply)
  + ipv6_cidr_block_association_id              = (known after apply)
  + ipv6_native                                 = false
  + map_public_ip_on_launch                     = true
  + owner_id                                    = (known after apply)
  + private_dns_hostname_type_on_launch         = (known after apply)
  + tags                                        = {
      + "Name" = "projet02-infra-subnet-public"
    }
  + tags_all                                    = {
      + "Name" = "projet02-infra-subnet-public"
    }
  + vpc_id                                      = (known after apply)
}

# aws_vpc.main will be created
+ resource "aws_vpc" "main" {
  + arn                                 = (known after apply)
  + cidr_block                          = "10.0.0.0/16"
  + default_network_acl_id              = (known after apply)
  + default_route_table_id              = (known after apply)
  + default_security_group_id           = (known after apply)
  + dhcp_options_id                     = (known after apply)
  + enable_dns_hostnames                = true
  + enable_dns_support                  = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                                  = (known after apply)
  + instance_tenancy                    = "default"
  + ipv6_association_id                 = (known after apply)
  + ipv6_cidr_block                     = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id                 = (known after apply)
  + owner_id                            = (known after apply)
  + tags                                = {
      + "Environment" = "dev"
      + "Name"        = "projet02-infra-vpc"
    }
  + tags_all                            = {
      + "Environment" = "dev"
      + "Name"        = "projet02-infra-vpc"
    }
}

Plan: 8 to add, 0 to change, 0 to destroy.
```



```
styve@Landry-SK:~/projet02-infra-durcissement/terraform$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
the following symbols:
  + create

Terraform will perform the following actions:

  # aws_internet_gateway.main will be created
  + resource "aws_internet_gateway" "main" {
      + arn      = (known after apply)
      + id       = (known after apply)
      + owner_id = (known after apply)
      + tags     = {
          + "Name" = "projet02-infra-igw"
        }
      + tags_all = {
          + "Name" = "projet02-infra-igw"
        }
      + vpc_id   = (known after apply)
    }

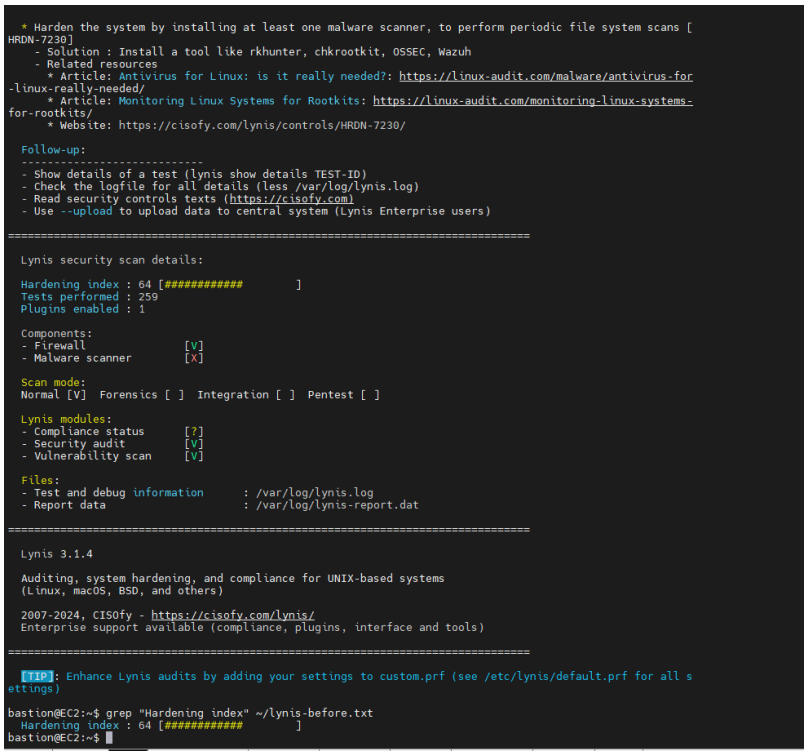
  # aws_route_table.public will be created
  + resource "aws_route_table" "public" {
      + arn             = (known after apply)
      + id              = (known after apply)
      + owner_id        = (known after apply)
      + propagating_vgws = (known after apply)
      + route           = [
          + {
              + cidr_block                = "0.0.0.0/0"
              + gateway_id                = (known after apply)
              # (11 unchanged attributes hidden)
            },
        ]
      + tags            = {
          + "Name" = "projet02-infra-rt-public"
        }
      + tags_all        = {
          + "Name" = "projet02-infra-rt-public"
        }
      + vpc_id          = (known after apply)
    }

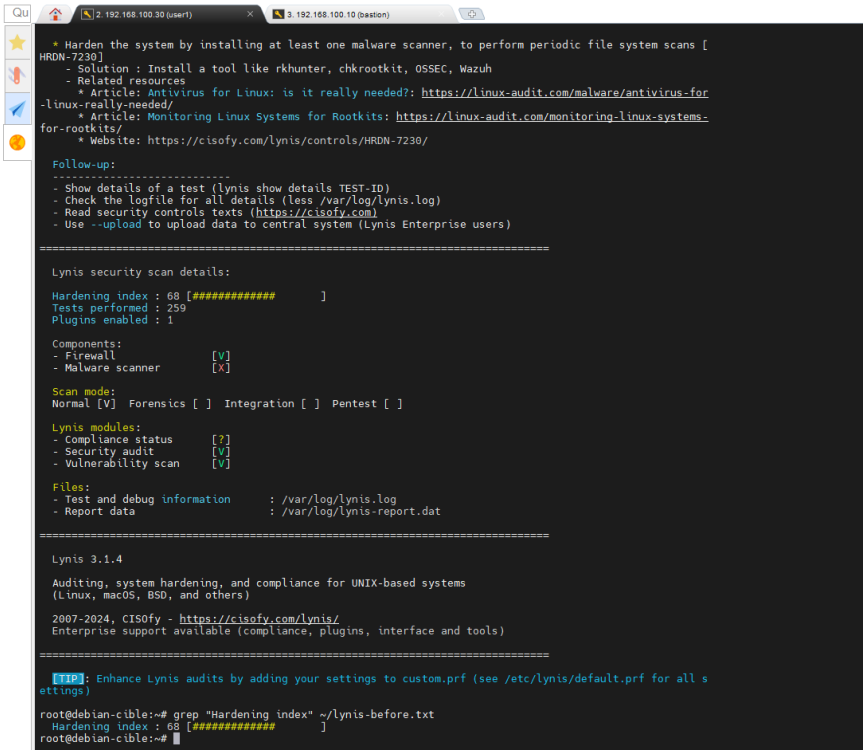
  # aws_route_table_association.public will be created
  + resource "aws_route_table_association" "public" {
      + id             = (known after apply)
      + route_table_id = (known after apply)
      + subnet_id      = (known after apply)
    }

  # aws_security_group.bastion will be created
  + resource "aws_security_group" "bastion" {
      + arn                    = (known after apply)
      + description            = "Security group bastion SSH"
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
```

Phase 2 — Conformité AVANT durcissement (baseline Lynis)

Avant toute action, un audit Lynis établit le point de départ de chaque serveur.





Phase 3 — Application du durcissement CIS (Ansible)

Le playbook, lancé depuis le bastion (poste de pilotage dual-homed), applique l'ensemble des règles CIS sur les deux serveurs. Premier passage : 27 tâches ok, 7 changed.

```
TASK [firewall-rules : CIS 3.5.x - Politique par defaut : autoriser la sortie] *************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Appliquer les regles autorisees de la zone] *************************
ok: [interne] => (item={'rule': 'allow', 'port': '22', 'proto': 'tcp', 'from_ip': '192.168.100.0/24', 'co
mment': 'SSH depuis DMZ uniquement'})
ok: [bastion] => (item={'rule': 'allow', 'port': '22', 'proto': 'tcp', 'comment': 'SSH admin'})
ok: [bastion] => (item={'rule': 'allow', 'port': '80', 'proto': 'tcp', 'comment': 'HTTP'})
ok: [bastion] => (item={'rule': 'allow', 'port': '443', 'proto': 'tcp', 'comment': 'HTTPS'})

TASK [firewall-rules : CIS 3.5.x - Activer la journalisation du pare-feu] ******************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Activer ufw] ********************************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.1 - Installer auditd] ****************************************************
changed: [bastion]
changed: [interne]

TASK [audit-logging : CIS 4.1.2 - Activer auditd au demarrage] *****************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.3-4.1.16 - Deployer les regles d'audit CIS] ******************************
changed: [bastion]
changed: [interne]

TASK [audit-logging : CIS 4.2.1.x - Rendre les journaux systemd persistants] ***************************
changed: [bastion]
changed: [interne]

TASK [audit-logging : CIS 4.2.1.x - Compresser les journaux volumineux] ********************************
changed: [bastion]
changed: [interne]

RUNNING HANDLER [audit-logging : restart auditd] *******************************************************
changed: [bastion]
changed: [interne]

RUNNING HANDLER [audit-logging : restart journald] *****************************************************
changed: [bastion]
changed: [interne]

PLAY RECAP *********************************************************************************************
bastion                    : ok=29   changed=7    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
interne                    : ok=29   changed=7    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

Phase 4 — Idempotence (critère noté — 20 pts)

Un code as-code doit être idempotent : le relancer ne doit plus rien modifier. Le 2e passage a révélé un `changed=1` — une tâche réseau (sysctl) se redéclarait à chaque exécution.

```
TASK [firewall-rules : CIS 3.5.x - Activer la journalisation du pare-feu] ******************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Activer ufw] ********************************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.1 - Installer auditd] ****************************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.2 - Activer auditd au demarrage] *****************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.3-4.1.16 - Deployer les regles d'audit CIS] ******************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.2.1.x - Rendre les journaux systemd persistants] ***************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.2.1.x - Compresser les journaux volumineux] ********************************
ok: [bastion]
ok: [interne]

PLAY RECAP *********************************************************************************************
bastion                    : ok=27   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    igno
red=0
interne                    : ok=27   changed=1    unreachable=0    failed=0    skipped=0    rescued=0    igno
red=0
```

Correctif : la section sysctl a été convertie en écriture d'un fichier de conf dédié (`/etc/sysctl.d/99-cis-hardening.conf`), idempotent au caractère près. Résultat : **changed=0** sur les deux serveurs.

```
TASK [firewall-rules : CIS 3.5.x - Installer ufw] ************************************************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Politique par defaut : tout refuser en entree] ***************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Politique par defaut : autoriser la sortie] ******************************
ok: [interne]
ok: [bastion]

TASK [firewall-rules : CIS 3.5.x - Appliquer les regles autorisees de la zone] ******************************
ok: [bastion] => (item={'rule': 'allow', 'port': '22', 'proto': 'tcp', 'comment': 'SSH admin'})
ok: [interne] => (item={'rule': 'allow', 'port': '22', 'proto': 'tcp', 'from_ip': '192.168.100.0/24', 'commen
t': 'SSH depuis DMZ uniquement'})
ok: [bastion] => (item={'rule': 'allow', 'port': '80', 'proto': 'tcp', 'comment': 'HTTP'})
ok: [bastion] => (item={'rule': 'allow', 'port': '443', 'proto': 'tcp', 'comment': 'HTTPS'})

TASK [firewall-rules : CIS 3.5.x - Activer la journalisation du pare-feu] **********************************
ok: [bastion]
ok: [interne]

TASK [firewall-rules : CIS 3.5.x - Activer ufw] ************************************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.1 - Installer auditd] ********************************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.2 - Activer auditd au demarrage] *********************************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.1.3-4.1.16 - Deployer les regles d'audit CIS] **********************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.2.1.x - Rendre les journaux systemd persistants] *******************************
ok: [bastion]
ok: [interne]

TASK [audit-logging : CIS 4.2.1.x - Compresser les journaux volumineux] ************************************
ok: [bastion]
ok: [interne]

PLAY RECAP *************************************************************************************************
bastion                    : ok=27   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    igno
red=0
interne                    : ok=27   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    igno
red=0
```

Phase 5 — Conformité APRÈS + delta chiffré

```
  Lynis security scan details:

  Hardening index : 70 [##############      ]
  Tests performed : 265
  Plugins enabled : 1

  Components:
  - Firewall               [V]
  - Malware scanner        [X]

  Scan mode:
  Normal [V]  Forensics [ ]  Integration [ ]  Pentest [ ]

  Lynis modules:
  - Compliance status      [?]
  - Security audit         [V]
  - Vulnerability scan     [V]

  Files:
  - Test and debug information      : /var/log/lynis.log
  - Report data                     : /var/log/lynis-report.dat

================================================================================

  Lynis 3.1.4

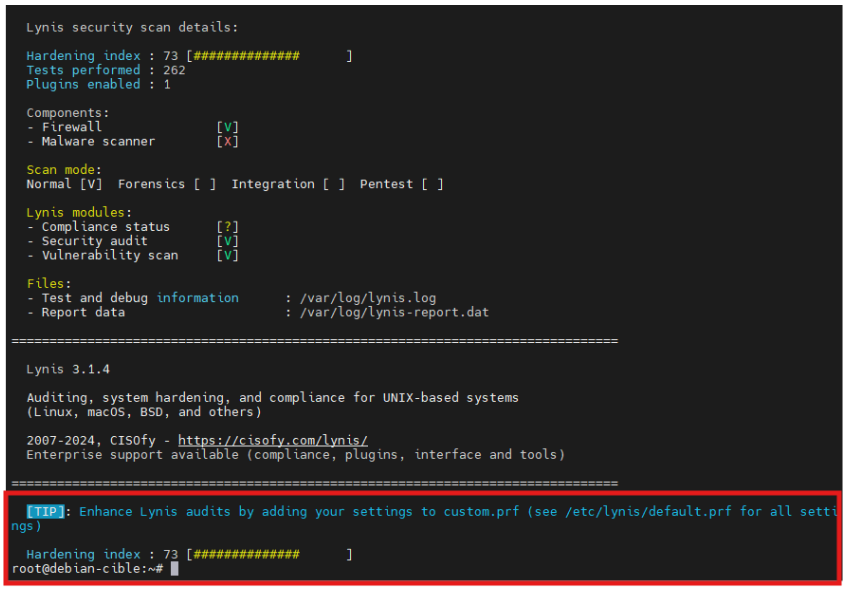
  Auditing, system hardening, and compliance for UNIX-based systems
  (Linux, macOS, BSD, and others)

  2007-2024, CISOfy - https://cisofy.com/lynis/
  Enterprise support available (compliance, plugins, interface and tools)

================================================================================

  [TIP]: Enhance Lynis audits by adding your settings to custom.prf (see /etc/lynis/default.prf for all setti
ngs)

  Hardening index : 70 [##############      ]
bastion@EC2:~/02-infra-durcie-terraform-ansible/ansible$
```



Synthèse Lynis avant / après

Serveur	Zone	Avant	Après	Gain
bastion (192.168.100.10)	DMZ	64 / 100	**70 / 100**	**+6**
interne (192.168.100.30)	interne	68 / 100	**73 / 100**	**+5**

Lynis 3.1.4 — rapports complets dans reports/

4. Segmentation réseau — tests de flux (25 pts)

Le serveur interne n'accepte le SSH **que depuis le réseau de la DMZ** ; tout le reste est bloqué. Le pare-feu ufw le confirme :

```
bastion@EC2:~/02-infra-durcie-terraform-ansible/ansible$ ping -c2 192.168.100.30
ssh user1@192.168.100.30 'echo OK; sudo ufw status verbose'
PING 192.168.100.30 (192.168.100.30) 56(84) bytes of data.
64 bytes from 192.168.100.30: icmp_seq=1 ttl=64 time=0.446 ms
64 bytes from 192.168.100.30: icmp_seq=2 ttl=64 time=0.616 ms

--- 192.168.100.30 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1018ms
rtt min/avg/max/mdev = 0.446/0.531/0.616/0.085 ms
OK
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    192.168.100.0/24           # SSH depuis DMZ uniquement
```

Tests depuis le bastion

Test	Commande	Résultat
Flux autorisé	`nc -zv 192.168.100.30 22`	**succeeded ✔️**
Flux bloqué	`nc -zv -w3 192.168.100.30 80`	**timed out ✔️**
Scan de ports	`nmap -Pn 192.168.100.30`	**seul 22/tcp open ✔️**

```
bastion@EC2:~$ nc -zv 192.168.100.30 22
Connection to 192.168.100.30 22 port [tcp/ssh] succeeded!
bastion@EC2:~$ nc -zv -w3 192.168.100.30 80     # doit echouer (timed out)
nmap 192.168.100.30             # seul 22/tcp "open", tout le reste "filtered"
Connection to 192.168.100.30 80 port [tcp/http] succeeded!
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-28 15:30 CET
Nmap scan report for 192.168.100.30
Host is up (0.0011s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

Nmap done: 1 IP address (1 host up) scanned in 5.81 seconds
bastion@EC2:~$
```

5. Découverte : Docker contourne ufw

Lors des tests, le port 80 de l'interne est resté ouvert **malgré ufw** : un conteneur Docker (DVWA, hérité du lab précédent) publiait le port 80. **Docker insère ses propres règles iptables en amont d'ufw**, contournant le pare-feu — un piège classique en production.

Correctif appliqué (un serveur interne durci ne doit pas héberger de service vulnérable exposé) : arrêt du conteneur et désactivation de Docker → port 80 fermé, nmap ne montre plus que le 22. En production : utiliser `ufw-docker` ou publier les ports sur `127.0.0.1` uniquement.

```
root@debian-cible:~# sudo ss -tlnp | grep ':80'
LISTEN 0      4096         0.0.0.0:80         0.0.0.0:*    users:(("docker-proxy",pid=1160,fd=8))
LISTEN 0      4096            [::]:80            [::]:*    users:(("docker-proxy",pid=1168,fd=8))
root@debian-cible:~# sudo docker ps
sudo docker stop $(sudo docker ps -q)
CONTAINER ID   IMAGE                 COMMAND      CREATED       STATUS       PORTS
       NAMES
cbe73c974367   vulnerables/web-dvwa   "/main.sh"   4 weeks ago   Up 3 hours   0.0.0.0:80->80/tcp, [::]:80->80
/tcp   dvwa
cbe73c974367
root@debian-cible:~# sudo systemctl disable --now docker docker.socket
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install disable docker
Removed '/etc/systemd/system/multi-user.target.wants/docker.service'.
Removed '/etc/systemd/system/sockets.target.wants/docker.socket'.
Disabling 'docker.service', but its triggering units are still active:
docker.socket
root@debian-cible:~# sudo ss -tlnp | grep ':80' || echo "port 80 ferme"
port 80 ferme
root@debian-cible:~# nmap 192.168.100.30
nc -zv -w3 192.168.100.30 80
-bash: nmap: command not found
cible-dvwa.lab [192.168.100.30] 80 (http) : Connection refused
root@debian-cible:~#
```

6. Durcissement CIS appliqué

Chaque tâche Ansible référence son numéro de règle CIS Debian 12, répartie sur 3 rôles.

Domaine	Règles CIS	Rôle
SSH (root, mot de passe, MaxAuthTries, X11, timeout, bannière)	5.1.x	hardening-base
Permissions des fichiers de comptes (passwd / shadow…)	6.1.x	hardening-base
Paramètres réseau noyau (redirects, martians, syncookies, rp_filter)	3.3.x	hardening-base
Mises à jour de sécurité automatiques	1.2.x	hardening-base
Politique de mots de passe (pwquality)	5.3.1	hardening-base
Systèmes de fichiers inutiles désactivés	1.1.1.x	hardening-base
Pare-feu ufw par zone (deny par défaut)	3.5.x	firewall-rules

Domaine	Règles CIS	Rôle
Audit système (auditd + règles)	4.1.x	audit-logging
Journalisation persistante	4.2.1.x	audit-logging

Couverture du barème (100 pts)

Critère	Points	Statut
Segmentation réseau testée	**25**	**Atteint ✔️**
Idempotence du code IaC	**20**	**Atteint — changed=0 ✔️**
Couverture CIS numérotée	**20**	**Atteint ✔️**
Delta de conformité Lynis	**20**	**Atteint — +6 / +5 ✔️**
Documentation reproductible	**15**	**Atteint ✔️**

7. Compétences démontrées

- Durcissement CIS automatisé et idempotent (Ansible, rôles)
- Segmentation réseau et pare-feu par zone (ufw, host-based)
- Audit système et journalisation (auditd, journald)
- Mesure de conformité objective (Lynis, delta chiffré avant/après)
- Architecture DMZ / bastion (jump host dual-homed)
- Compréhension des interactions Docker / pare-feu (sécurité de production)
- Infrastructure as Code (Ansible + Terraform AWS)
- Documentation reproductible et adaptation aux contraintes réelles

---

Projet réalisé en autonomie — Styve Landry SIME KOUNOU — styvelandry1@outlook.fr — octobre 2026